

用 AI 开发，记住这几点要素

1、遇到你无法准确描述、无法让它理解的东西，就给参考物。

不要总觉得“它应该懂我在说什么”。很多时候你脑子里有画面，但 AI 没有。

比如你想要某种页面风格、某种交互方式、某个组件结构、某种业务流程，如果你自己讲不清，就直接给它参考链接、截图、现成产品、开源项目、设计稿、接口文档。

如果你手上没有参考物，那就直接让它联网去搜，先找到类似案例，再基于案例来做，而不是凭空脑补。

AI 最怕的不是难题，而是模糊题。

2、经常让它去看官方技术文档、开发文档，不要让它自己猜。

这一点特别重要。

很多人用 AI 开发，最容易犯的错就是一句“你帮我改一下”，然后让它靠记忆乱推。

但只要涉及框架 API、第三方库、部署方式、配置项、版本差异、官方推荐写法，就应该优先让它查官方文档。

因为 AI 很容易“看起来很懂”，但它给你的可能是旧版本写法、错误参数、已经废弃的方案，甚至是它自己拼出来的“像是对的东西”。

所以你要养成一个习惯：

凡是技术细节、框架行为、库的用法、报错处理，优先看官方文档，不要凭感觉修。

3、对话一定要目标明确，不要只给一个模糊概念。

你不能跟它说：

- “你帮我优化一下。”
- “你看看还能不能再改好一点。”
- “你弄得高级一点。”

这种话对 AI 来说空间太大了，它会自己发挥，而且经常越改越偏。

你应该说清楚：

- 你到底要改什么
- 改到什么程度
- 哪些地方不能动
- 你的验收标准是什么
- 做完什么算结束

比如不是“优化一下登录页”，而是：

“请优化登录页的首屏加载速度，只允许处理图片加载和接口请求顺序，不要改 UI 结构，不要改接口协议，最终目标是首屏白屏时间缩短，并保证现有登录逻辑不变。”

这种表达，AI 才能真正帮你做事。

4、不要用你主观模糊的话去强加给它，不要总说‘我觉得应该这样’。

- “我觉得这里怪怪的。”
- “我感觉这样不高级。”
- “我觉得这块可能有问题。”

这种说法不是不能用，但如果只有主观判断，没有客观依据，AI 往往会被你带偏。

它会顺着你的感觉继续编理由、编方向，最后越改越离谱。

更好的说法是把主观感觉翻译成客观问题：

- 是视觉层级不清楚？
- 是交互流程太长？
- 是响应速度慢？
- 是代码耦合高？
- 是组件复用性差？
- 是报错没有兜底？

你要尽量把“感觉”变成“问题定义”。

不要把情绪丢给 AI，要把约束丢给 AI。

5、不要让它一步到位，先让它分析，再让它执行。

很多人一上来就是：

- “直接改。”
- “直接重构。”
- “直接给我做完。”

这样很容易翻车。

更稳的方式是分两步：

1. 第一步，先让它分析问题、列方案、讲风险。
2. 第二步，你确认方向后，再让它执行。

尤其是项目已经能跑的时候，这个习惯特别重要。

因为 AI 一旦进入“自我发挥模式”，很可能开始大改、乱抽象、过度重构，最后把本来能用的东西搞复杂。

所以先审查、再动手，能挡掉很多无意义折腾。

6、一定要给边界，不然它会一直做下去。

AI 最大的问题之一，不是不会做，而是不知道什么时候该停。

特别是你说“再优化一下”“看看还有没有可以改的”，它就可能一直往下挖。

今天优化命名，明天优化结构，后天优化抽象，没完没了。

所以你要明确告诉它：

- 只做前 2 个最高收益问题
- 不要改架构
- 不要做大规模重构
- 做完就停
- 没有我确认不要继续下一轮

没有停止条件的任务，AI 默认会一直往前拱。

7、改任何东西之前，让它先说清楚：为什么改、改哪里、风险是什么。

这一点非常关键。

你不能只看它“改完了”，你还要知道它“为什么这么改”。

否则你最后会发现代码虽然变了，但你根本不知道：

- 它改动是否必要
- 有没有引入副作用
- 有没有误伤现有逻辑
- 为什么不选别的方案

所以最好的方式是让它在动手前先输出：

- 问题描述
- 原因分析
- 方案选择
- 风险点
- 验证方式

这样你就不会被它牵着走，而是你在控制它。

8、不要默认它理解你的业务，上下文不够时要主动补。

AI 对代码理解得再快，也不代表它真的懂你的业务。

它可能知道函数怎么写、组件怎么拆，但它不一定知道：

- 哪个流程是核心链路
- 哪个字段不能动
- 哪个页面是临时方案
- 哪个逻辑是为了兼容旧数据
- 哪个 bug 其实不是 bug，而是业务要求

所以一旦涉及业务规则、历史包袱、兼容逻辑，就要明确告诉它。

不然它会把“看起来不合理但实际上必须存在”的东西当成脏代码清掉。

9、遇到报错，不要只贴一句报错，让它一起看上下文。

AI 修 bug 很强，但前提是你给的信息够完整。

不要只用一句：

“这里报错了，帮我看看。”

正确做法应该一起给：

- 报错信息
- 相关代码
- 调用链路
- 运行环境
- 复现步骤
- 最近改过什么

因为很多 bug 不是光看错误文本就能定位的。

错误提示只是结果，上下文才是原因。

10、不要让它瞎试，要让它先缩小问题范围。

很多时候 AI 一收到 bug，就开始这里改一下那里试一下。

这样非常容易进入“碰运气修 bug”的状态。

更好的方式是让它先判断：

- 问题更可能出在哪一层
- 哪几种原因概率最高
- 应该先验证什么
- 哪些东西先不要动

也就是先定位，再修复。

如果定位都没做清楚，它的修改很可能只是随机命中。

11、对 AI 的要求要尽量可验证，不要只追求‘看起来不错’。

比如你让它优化性能，那就说清楚：

- 优化哪一段
- 看什么指标
- 接受什么结果
- 用什么方式验证

比如你让它改页面，不要只说“更好看一点”，而是说：

- 信息层级更清晰
- 首屏 CTA 更突出
- 表单填写路径更短
- 移动端不折行错位

越能验证，越不容易跑偏。

12、不要把 AI 当神，也不要把它当纯工具，要把它当一个执行力很强但需要管理的协作者。

它的优点是快、广、能连续产出；

它的缺点是会脑补、会跑偏、会过度自信、会把不确定说得像确定。

所以真正高效的用法不是“全交给它”，也不是“啥都不信它”，而是：

让它干活，但你来定目标、定边界、定标准、定停点。

这样它才会变成生产力；否则很容易变成新的混乱来源。

最后总结

AI 开发不是看它会不会写代码，而是看你会不会给任务、给边界、给标准。

你描述得越清楚，它越像工程师；你描述得越模糊，它越像算命先生。